

Java Integer Literals & Bitwise Operations

Binary, octal, decimal, hex prefixes — and how bitwise ops work underneath

1. Number Literal Prefixes

Java lets you write integer literals in four bases. The prefix tells the compiler how to read the digits that follow — it has no effect once compiled.

Base	Prefix	Example	Digits allowed
Binary (base 2)	0b or 0B	0b1010	0, 1
Octal (base 8)	0 (leading zero)	012	0–7
Decimal (base 10)	none	10	0–9
Hexadecimal (base 16)	0x or 0X	0xA	0–9, a–f

Why the naming

- 0x → “x” for hexadecimal.
- 0b → “b” for binary.
- Octal has no letter — it's just a bare leading 0. This is an old convention Java copied from C, where octal was added early and never got its own letter. It's a common gotcha: writing 012 expecting “twelve” actually gives you 10 in decimal.

Invalid digits are a compile error

Binary literals may only contain 0 and 1 after the 0b prefix. Writing 0b2 (or any digit 2–9) fails to compile:

```
error: illegal digit
```

2. What's Actually Stored in Memory

Once compiled, 0b1010, 10, 0xA, and 012 (octal 12 = decimal 10) are all exactly the same bits in memory. There is no tag on the value saying “I’m binary” or “I’m decimal” — that distinction exists only in your source code, never in the running program.

```
long x = 0b1010L;  
System.out.println(x);    // prints 10
```

It prints 10 not because the number “is” decimal internally, but because `println()` (via `Long.toString()`) always formats integers as decimal strings by default. That's a choice made by that method, not a property of the stored value.

To see the same value in a different base, ask for it explicitly:

```
System.out.println(Long.toBinaryString(x)); // prints 1010
System.out.println(Long.toHexString(x));   // prints a
System.out.println(Long.toOctalString(x));  // prints 12
```

Same underlying value, four different string representations — the format is purely a printing/display choice.

3. long Capacity & BigInteger

A long is a 64-bit signed integer. Its maximum value is 9,223,372,036,854,775,807 — about 19 decimal digits. A 30-digit decimal number overflows it.

But 30 *binary digits (bits)* is a completely different story: 30 bits fit easily inside a 64-bit long, with 34 bits to spare.

```
long binaryBits = 0b101010101010101010101010101010L; // fits perfectly

// A 30-digit DECIMAL number, however, needs BigInteger:
BigInteger big = new BigInteger("123456789012345678901234567890");
```

4. Bitwise Operators Work Normally

A number written with 0b behaves exactly like any other long once compiled — it can be added, multiplied, compared, and used with bitwise operators. In fact, bitwise operators are especially natural here because they map directly onto the bit pattern.

Operator	Name	Behavior
&	AND	Keeps a 1 only where both operands have a 1
	OR	Keeps a 1 where either operand has a 1
^	XOR	Keeps a 1 where the bits differ
~	NOT	Flips every bit (0→1, 1→0)
<< / >>	Shift	Slides bits left/right, filling with 0s

Full example

```
public class BinaryExample {
    public static void main(String[] args) {
        // A 30-digit binary literal
        long binaryBits = 0b101010101010101010101010101010L;

        // A mask to test the last bit
        long mask = 0b1L;

        // Bitwise AND
        long resultAnd = binaryBits & mask;
```

```
// Shift left by 2 (slides bits, pads with 0s)
long shifted = binaryBits << 2;

// Print as binary to see the actual bit pattern
System.out.println("Original: " + Long.toBinaryString(binaryBits));
System.out.println("Shifted:  " + Long.toBinaryString(shifted));
    }
}
```

Quick summary: the 0b/0/0x prefixes only affect how the compiler parses your source text. Once compiled, all four bases produce the identical bit pattern in memory, and every operator — arithmetic or bitwise — works on that bit pattern the same way regardless of which prefix you used to write it.